

Assignment 01 - Pool Game

Programmazione Concorrente e Distribuita

Lorenzo / LoryBug

2026-07-07

1. Analisi del problema

L'obiettivo dell'assignment è progettare e implementare una versione concorrente del gioco Pool. Il gioco contiene un piano bidimensionale con molte palle piccole, due palle dei giocatori (umano e bot), due buche negli angoli superiori, collisioni elastiche, attrito, punteggio e condizioni di fine partita.

Gli aspetti rilevanti dal punto di vista concorrente sono i seguenti:

- **Reattività dell'input:** il giocatore umano deve poter premere UP, DOWN, LEFT, RIGHT senza bloccare il ciclo fisico del gioco.
- **Bot asincrono:** il bot deve giocare indipendentemente dall'utente e dal frame rate.
- **Rendering Swing:** la GUI deve essere aggiornata rispettando il vincolo dell'EDT (Event Dispatch Thread).
- **Stato condiviso:** palle, punteggi, stato della partita e buffer dei comandi sono dati condivisi tra componenti attivi.
- **Prestazioni:** il numero di small balls può essere elevato; la fase critica è il controllo collisioni small-small, che nello sketch originale è $O(n^2)$.

La soluzione è stata sviluppata in due versioni, come richiesto dalla consegna:

- `pcd.ass01.multithreaded.PoolMultithreaded`: versione basata su platform thread Java.
- `pcd.ass01.taskbased.PoolTaskBased`: variante task-based basata su `ExecutorService`.

2. Design adottato

2.1 Struttura dei package

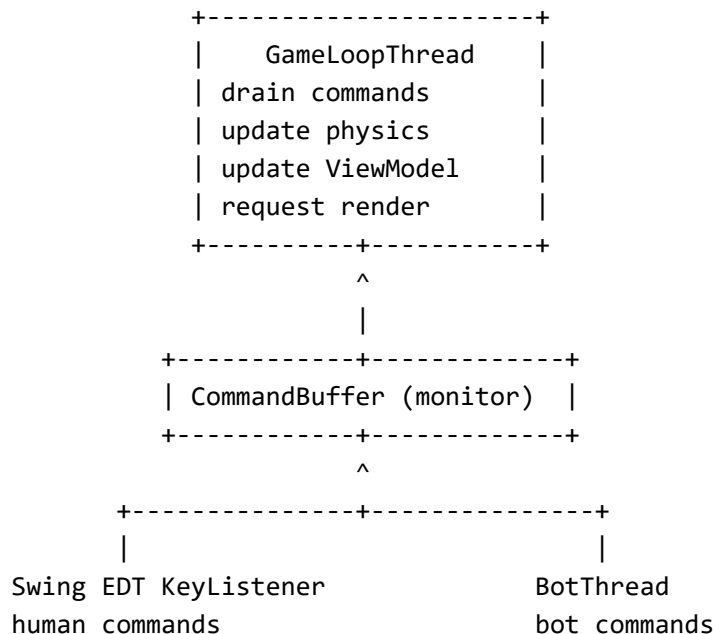
```
src/pcd/ass01/  
  common/          model, snapshot, comandi, buffer monitor
```

view/	GUI Swing, ViewModel, RenderSynch
multithreaded/	GameLoopThread, BotThread, main multithreaded
taskbased/	TaskBasedBoard, main task-based
benchmark/	benchmark headless per performance test

Il package common contiene gli elementi riusati da entrambe le versioni:

- GameModel: interfaccia comune del model.
- Board: model della versione multithreaded.
- TaskBasedBoard: model task-based, nel package taskbased.
- Ball, Vec2, Bounds, Hole: fisica e geometria.
- CommandBuffer: monitor produttore-consumatore non bloccante per la EDT.
- GameCommand, KickHumanCommand, KickBotCommand: command pattern.
- GameSnapshot: snapshot immutabile usato dalla GUI.

2.2 Architettura concorrente



La GUI non modifica direttamente il Board. Le pressioni dei tasti producono comandi nel CommandBuffer. Il game loop drena i comandi disponibili a inizio frame e poi aggiorna la fisica.

Il bot e' un platform thread separato. Periodicamente controlla se la sua palla e' ferma e, in caso positivo, produce un comando KickBotCommand con impulso casuale. La consegna specifica che il bot non deve necessariamente essere intelligente, quindi questa strategia e' sufficiente.

2.3 Monitor e sincronizzazione

Sono usati costrutti high-level basati su monitor Java (synchronized):

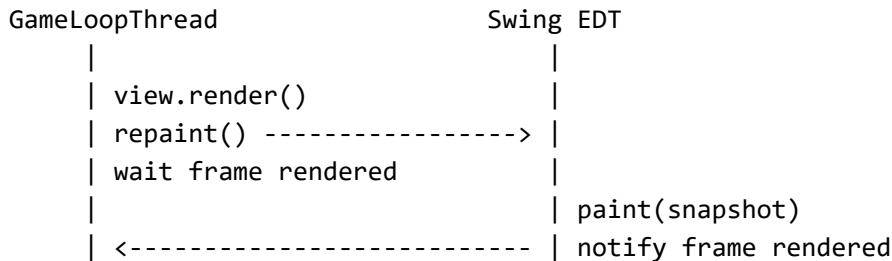
- Board e TaskBasedBoard: sincronizzano update, snapshot e comandi di kick.
- CommandBuffer: implementa un buffer thread-safe con offer e drainToList.
- RenderSynch: sincronizza il thread del game loop con l'EDT durante il rendering.

Il buffer comandi usa offer non bloccante: se il buffer e' pieno, la EDT non resta bloccata. Questo e' importante per mantenere reattiva la GUI.

2.4 Rendering e snapshot

La GUI Swing non legge mai lo stato mutabile del model. Il game loop costruisce un GameSnapshot e lo passa al ViewModel. Il frame Swing disegna solo lo snapshot, riducendo il rischio di race condition tra fisica e rendering.

Il rendering e' sincronizzato con RenderSynch:



3. Comportamento del gioco

3.1 Regole implementate

- Il giocatore umano controlla la palla blu con le frecce.
- Il bot controlla la palla arancione con impulsi casuali periodici.
- Le palline rimbalzano sui bordi e tra loro con collisioni elastiche.
- L'attrito riduce progressivamente la velocita'.
- Quando una small ball entra in buca viene rimossa.
- Il punteggio aumenta solo se la small ball e' entrata in buca dopo contatto diretto con una palla giocatore.
- Se una small ball spinge un'altra small ball in buca, il punto non viene assegnato, come richiesto dalla consegna.
- Se una palla giocatore entra in buca, vince l'altro giocatore.
- Se finiscono le small balls, vince chi ha il punteggio maggiore; in caso di parita' lo stato e' DRAW.

3.2 Gestione dell'ownership per il punteggio

Ogni small ball mantiene lastDirectTouch, con valori HUMAN, BOT, NONE. Una collisione diretta tra player ball e small ball assegna l'ownership al player. Una collisione small-small resetta l'ownership a NONE, per evitare di assegnare punti quando una small ball ne spinge un'altra in buca.

4. Petri net del comportamento

La seguente rete di Petri descrive il livello di astrazione principale del sistema: input, bot, game loop e rendering.

4.1 Posti

Posto	Significato
P_wait_input	EDT in attesa di eventi tastiera
P_cmd_buffer	Comandi disponibili nel buffer
P_loop_ready	Game loop pronto a iniziare un frame
P_physics	Fisica in aggiornamento
P_snapshot_ready	Snapshot pronto per rendering
P_rendering	EDT sta eseguendo paint
P_bot_wait	Bot in attesa del prossimo tiro
P_game_over	Partita terminata

4.2 Transizioni

Transizione	Input	Output	Significato
T_key_pressed	P_wait_input	P_cmd_buffer, P_wait_input	L'utente produce un comando
T_bot_kick	P_bot_wait	P_cmd_buffer, P_bot_wait	Il bot produce un comando
T_start_frame	P_loop_ready	P_physics	Inizio frame
T_drain_commands	P_cmd_buffer, P_physics	P_physics	Il game loop consuma i comandi
T_update_physics	P_physics	P_snapshot_ready	Aggiornamento fisico completato
T_repaint	P_snapshot_ready	P_rendering	Richiesta repaint alla EDT
T_frame_rendered	P_rendering	P_loop_ready	Rendering completato
T_end_game	P_physics	P_game_over	Fine partita

4.3 Osservazioni sulla rete

La rete evidenzia che input umano e bot possono produrre comandi indipendentemente dal game loop. Il game loop e' l'unico componente che applica i comandi allo stato fisico, quindi le modifiche al model sono serializzate a livello di frame. Il rendering e' una fase sincronizzata: il game loop non procede al frame successivo finche' l'EDT non ha completato il disegno dello snapshot.

5. Versione multithreaded

La versione multithreaded usa solo platform thread Java:

- `GameLoopThread`: thread principale della simulazione.
- `BotThread`: thread del giocatore automatico.
- `Swing EDT`: thread gestito da Swing per eventi e rendering.

Il game loop esegue:

```
while running:
    drain commands
    update board physics
    compute fps
    build snapshot
    render synchronously
    sleep until target frame period
```

Questa versione privilegia semplicità e correttezza: le collisioni sono calcolate in modo sequenziale all'interno del monitor del Board, evitando data race sulle palle.

6. Versione task-based

La versione task-based usa `ExecutorService` in `TaskBasedBoard`. La fase di update indipendente delle small balls viene divisa in chunk:

```
for each chunk of balls:
    submit task:
        update position, friction and boundary constraints
wait all futures
resolve collisions sequentially
collect balls in holes
check game over
```

La parallelizzazione è limitata alla fase sicura: posizione, attrito e vincoli sui bordi. Ogni task modifica palle diverse. Le collisioni non sono parallelizzate in questa versione perché `Ball.resolveCollision(a,b)` muta entrambe le palle; parallelizzare direttamente il doppio ciclo produrrebbe race condition quando due task toccano la stessa palla.

Questa scelta è coerente con i principi del corso: prima safety e correttezza, poi ottimizzazione. Una possibile estensione sarebbe introdurre lock ordinati per palla oppure una fase parallela di detection seguita da una fase seriale di apply.

7. Performance test

7.1 Metodologia

E' stato aggiunto un benchmark headless (`pcd.ass01.benchmark.PoolBenchmark`) che misura solo update fisico, senza GUI e senza input interattivo. Il benchmark confronta:

- model sequenziale usato dalla versione multithreaded (`Board`);
- model task-based (`TaskBasedBoard`).

Ambiente di test:

- JDK: OpenJDK 23.0.1.
- Maven: 3.9.11 tramite wrapper.
- Processori disponibili alla JVM: 8.

Comando usato:

```
.\mvnw.cmd exec:java "-Dexec.mainClass=pcd.ass01.benchmark.PoolBenchmark"
```

7.2 Risultati

Configurazione	Frame	Board totale (ms)	Task totale (ms)	Board avg/frame (ms)	Task avg/frame (ms)	Speedup
minimal	500	10	14	0.02	0.03	0.71
large	200	486	460	2.43	2.30	1.06
massive	20	5859	6110	292.95	305.50	0.96

7.3 Discussione

La versione task-based migliora leggermente nella configurazione large, dove il lavoro parallelo sull'update delle palline compensa il costo di scheduling dei task. Nella configurazione minimal, l'overhead dell'Executor e' superiore al lavoro utile, quindi la versione task-based e' piu' lenta.

Nella configurazione massive, la versione task-based non mostra speedup significativo. La ragione e' la legge di Amdahl: la fase $O(n^2)$ di collisione small-small resta seriale e domina il tempo totale. Parallelizzare solo l'update posizione/attrito non basta quando il collo di bottiglia e' quasi interamente nelle collisioni.

La conclusione e' che il task-based approach e' corretto ma il beneficio prestazionale e' limitato dalla parte seriale. Per sfruttare meglio gli 8 core disponibili servirebbe parallelizzare anche collision detection/resolution con una strategia sicura.

8. Verifica e JPF

La verifica completa della GUI Swing e della fisica continua non e' adatta a JPF: lo spazio degli stati sarebbe troppo grande e include componenti esterni complessi. La parte piu' sensata da verificare con model checking e' costituita dai monitor e dalle transizioni di stato ridotte.

Componenti candidati alla verifica JPF:

- `CommandBuffer`: nessuna perdita di comandi finche' il buffer non e' pieno, nessuna eccezione concorrente, assenza di deadlock.
- Transizioni del game state: da `RUNNING` a `HUMAN_WON`, `BOT_WON`, `DRAW`.
- Regola dello score: solo collisione diretta player-small assegna ownership; collisione small-small resetta a `NONE`.

Esempio di proprieta' safety da verificare:

```
scoreHuman >= 0
scoreBot >= 0
status in {RUNNING, HUMAN_WON, BOT_WON, DRAW}
not (status == HUMAN_WON and status == BOT_WON)
```

Esempio di scenario ridotto per JPF:

```
Thread 1: produce KickHumanCommand
Thread 2: produce KickBotCommand
Thread 3: drain commands and apply them
Property: no deadlock, buffer size never negative, commands applied
atomically
```

La verifica manuale tramite build e benchmark e' stata eseguita con:

```
.\mvnw.cmd test
.\mvnw.cmd package
```

Entrambi i comandi terminano con BUILD SUCCESS.

9. Come eseguire

Da `assignments/assignment-01`:

```
# Build
.\mvnw.cmd test

# Versione multithreaded
.\mvnw.cmd exec:java

# Versione task-based
.\mvnw.cmd -Ptaskbased exec:java
```

Configurazioni disponibili:

```
.\mvnw.cmd exec:java "-Dexec.args=minimal"  
.\mvnw.cmd exec:java "-Dexec.args=massive"  
  
.\mvnw.cmd -Ptaskbased exec:java "-Dexec.args=minimal"  
.\mvnw.cmd -Ptaskbased exec:java "-Dexec.args=massive"
```

10. Conclusioni

L'implementazione soddisfa le richieste principali dell'assignment: il gioco e' concorrente, usa platform thread nella prima versione, usa ExecutorService nella variante task-based, separa model/view/controller, usa monitor per lo stato condiviso e mantiene la GUI reattiva.

La parte piu' critica e' la fisica delle collisioni. La scelta implementata evita race condition e mantiene corretta la simulazione. I risultati prestazionali mostrano pero' che la parallelizzazione dell'update indipendente non basta per la configurazione massiva: il collo di bottiglia e' il doppio ciclo delle collisioni. Questo risultato e' coerente con la legge di Amdahl e indica chiaramente la direzione di miglioramento futura.